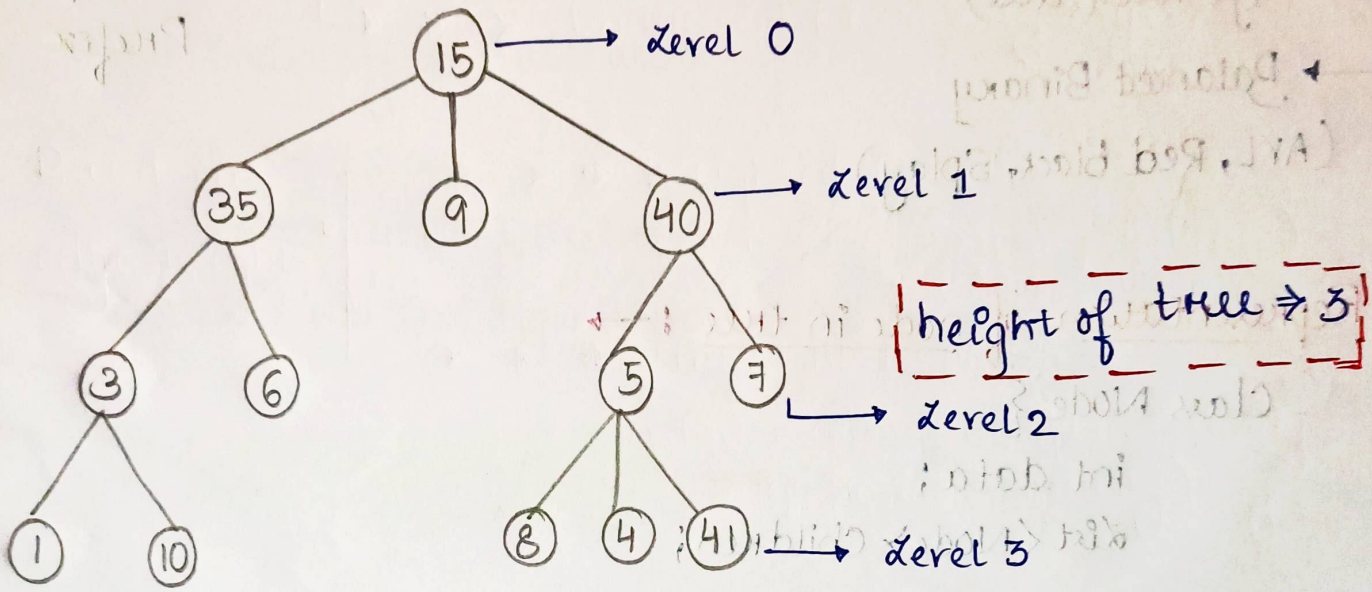
2) TREES :→

- hierarchial data structure
- used to organise represent data in a parent - child relationship.
- top most node ≡ root
- Non-linear data structure



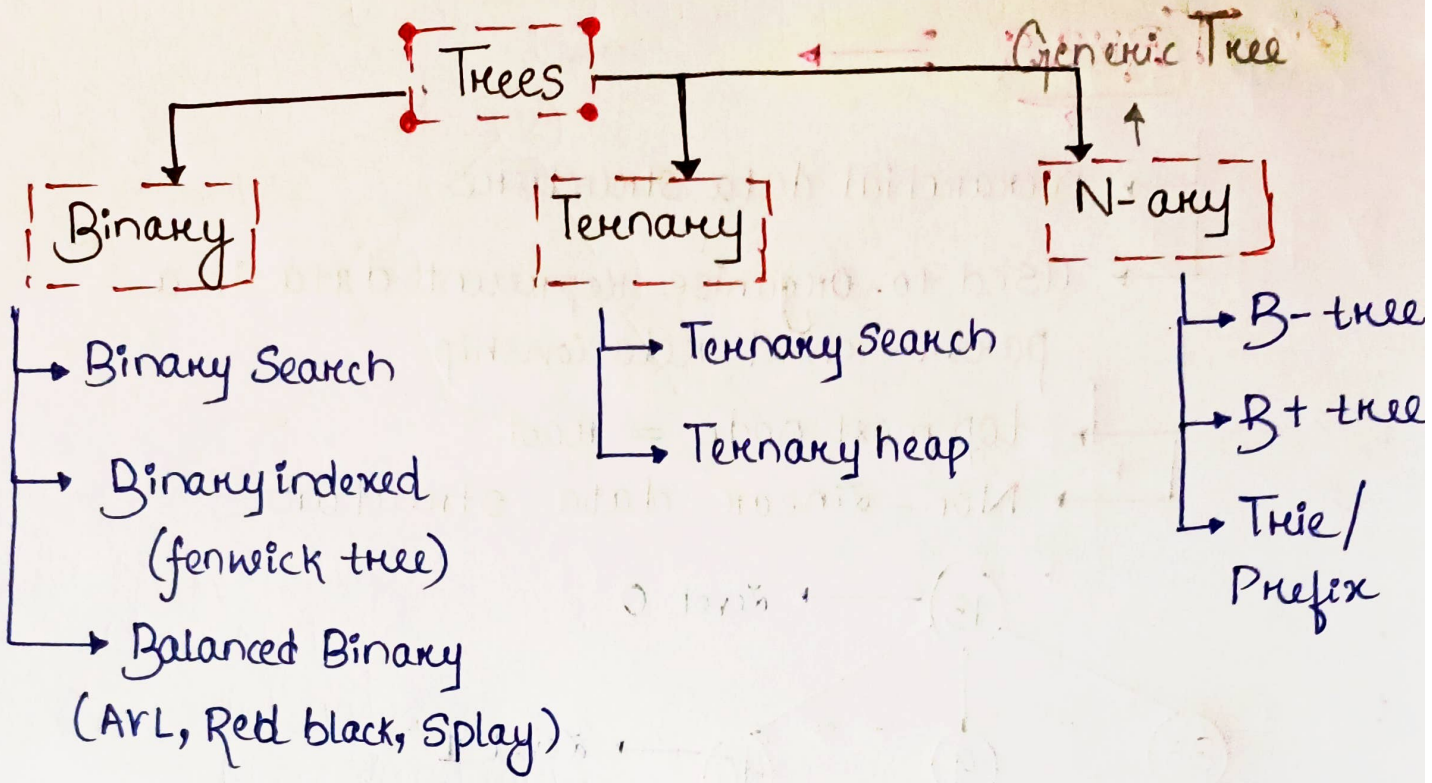
1) Leaf nodes :→ Nodes that do not have any children (external nodes)

2) Internal node :→ A node with at least one child

3) Degree of a node :→ no of subtrees attached to it

4) Depth of a node :→ Length of the path from the root to that node.

(*) A tree with N nodes will always have N-1 edges.



⊙ Representation of node in tree :→

```
Class Node {
        int data;
        List<Node> children;
        Node(int x) {
            data = x;
            children = new ArrayList();
        }
    }
```

⊙ Basic operations of tree: →

i) Create
ii) Insert
iii) Search
iv) Traversal [DFS, BFS]

Uses of tree :→

i) Hierarchial structure
ii) Searching efficiency
iii) Sorting
iv) Dynamic data
v) Efficient insertion and deletion
vi) Easy to implement
vii) for indexing in database
viii) Parsing
ix) Graphics and UI design
x) decision making algorithms in AI

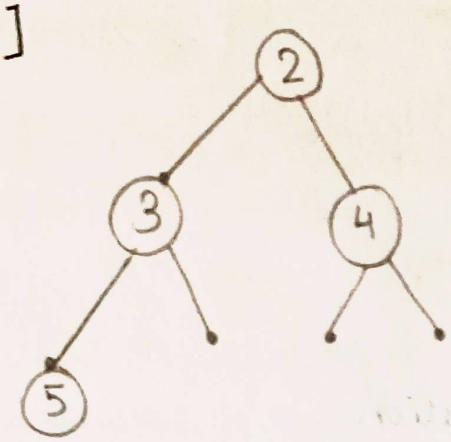
A) Binary Tree :→

↳ has two children

Q.) Create a node of a binary tree

```
class Node {
    int key;
    Node left, right;
    Public Node (int item)
    {
        key = item;
        left = right = null;
    }
}
```

Q> Write a code to create a binary tree with four nodes [2,3,4,5]



Node structure

```
Class Node {
    int data;
    Node left, right;
    Node (int d) {
        data = d
        left = null;
        right = null;
    }
}

class GFG {
    Public Static void main (String [] args) {
        // initialise and allocate memory for tree nodes
        Node firstnode = new Node (2);
        NOde Secondnode = new Node (3);
        Node Thirdnode = newNode (4);
        Node fourthnode = new Node (5);
        // connect binary tree nodes
        firstNode. left = secondNode;
        firstNode. right = Third Node;
        secondNode. left = fourthNode;
    }
}
```

Properties of Binary Tree :-

1) max no of nodes at level 'l' ⇒ 2^L
2) max no of nodes of height 'h' ⇒ $2^{h+1} - 1$
3) Least no of level with 'L' leaves ⇒ $\lceil \log_2 L \rceil + 1$
4) max height / max no of levels with 'N' nodes ⇒ $\log_2 N$
5) Total no of leaf nodes = total number of nodes with 2 children + 1.
6) Total edges of 'n' nodes ⇒ n-1 edges.

Advantages of Binary Tree

i) Efficient search
ii) Efficient memory

Disadvantages of Binary Tree

i) Limited structure
ii) Space inefficiency.

Types of Binary Tree :-

1. On the basis of number of children
   i) Full Binary tree [every node has 0 or 2 children]
   ii) Degenerate Binary tree [internal node has one child] [single child either left or right]
   iii) Skewed Binary tree
   [dominated by either left nodes or right nodes]

2. On the basis of level completion
   i) Complete Binary Tree [every level except last must be filled]
   ii) Perfect Binary Tree [Leaf node same level; every internal node had 2 child]
   iii) Balanced Binary Tree

Depth of node = |height of left child - height of right child|

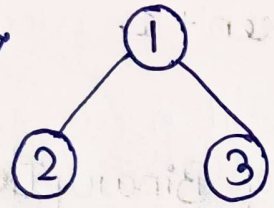
Inorder Traversal :→

Left → Root → Right

→ returns elements in sorted order.

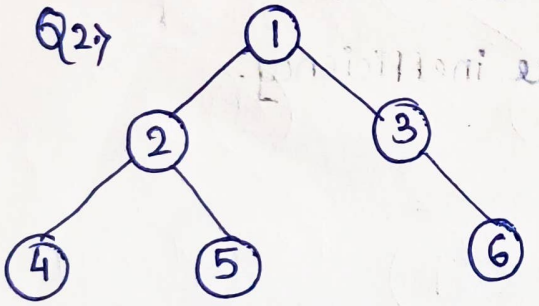
→ ensures nodes are processed in hierarchial sequence.

Q1)



Output : [2, 1, 3]

Q2)

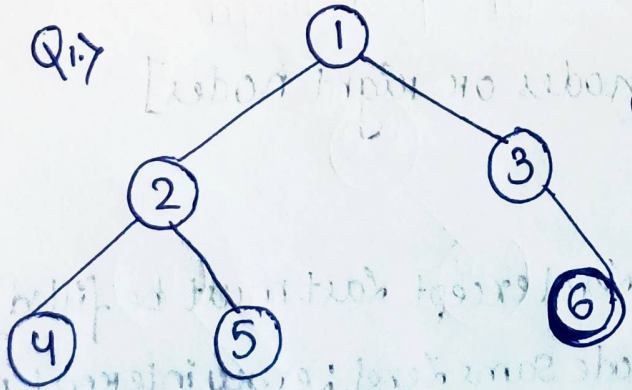


Output : [4, 2, 5, 1, 3, 6]

Preorder Traversal :→

Root → Left → Right

Q1)

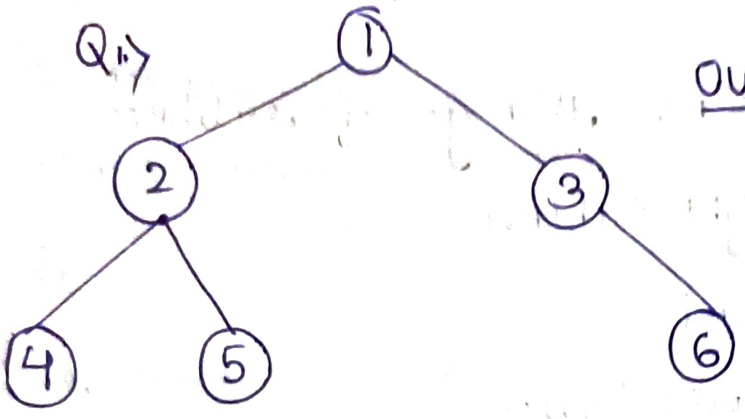


Output : [1, 2, 4, 5, 3, 6]

Post order Traversal :→

Left → Right → Root

Q.)



Output : [4, 5, 2, 6, 3, 1]

Binary Search Tree

i) → maintains its elements in sorted order. [inorder Traversal]

ii) all nodes in left subtree < node's value.

iii) all nodes in right subtree > nodes value.

iv) duplicates usually not allowed.

v) Average height :- O(log n) [for balanced BST]

vi) Worst case height :- O(n) [tree becomes skewed]

Searching in Binary search Tree :-

i) compare input key with root
   ↳ done if matches

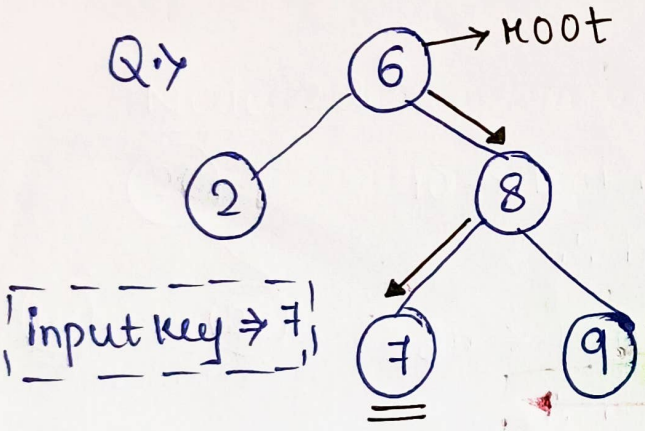
ii) If the value is smaller < root, then go left subtree

iv) If value is >, then go right subtree

v) repeat traversal until found

vi) return True if found otherwise false.

Q.)



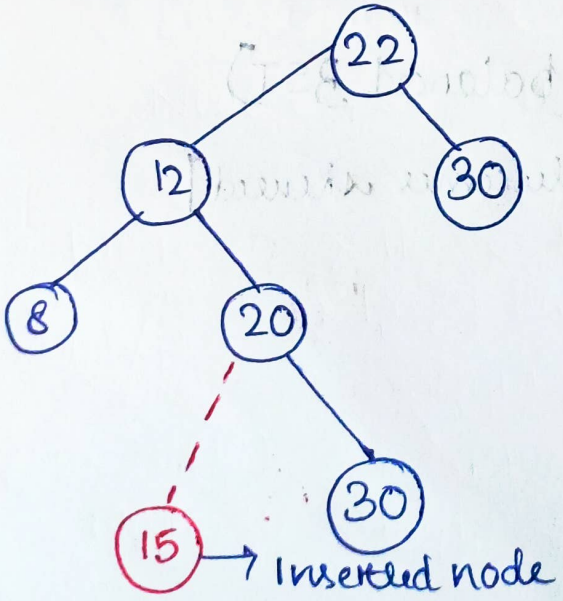
Output: 7 > 6
↳ go right

8 > 7
↳ go left subtree of 8

Hence, 7 is found.

Insertion in BST :-

input key = 15



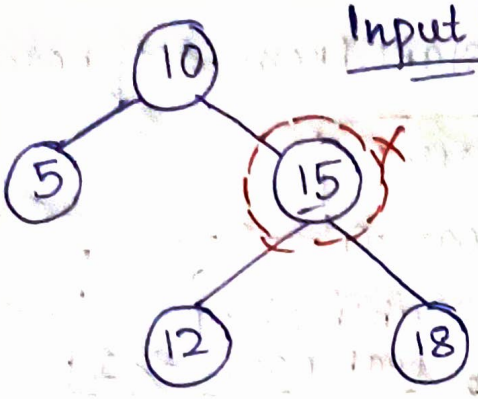
Output: 15 < 22
↳ go left

15 > 12
↳ go right

15 < 20
↳ go left

∴ Hence, node inserted

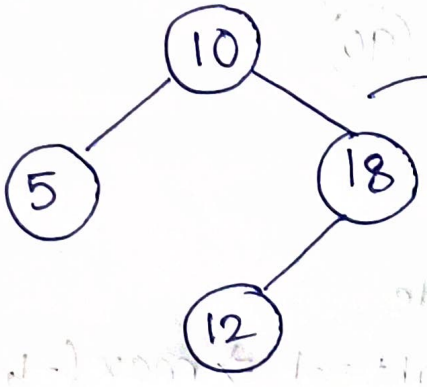
Deletion in BST :-



Input key :- 15 | to be deleted.

15 > 10
↳ go right (∵ found the key)

⇒ new BST



→ final output

AVL Tree :-

Root value < for right Subtree
Root value > for left Subtree.

↳ self Balancing BST

↳ Difference ⇒ | hgt of left subtree - hgt of right Subtree |
↓
don't exceed 1 for any node.

a) Balance Factor = | hgt of left subtree - right subtree |

b) For Balanced tree (for every node) = $-1 \leq$ Balanced Factor ≤ 1

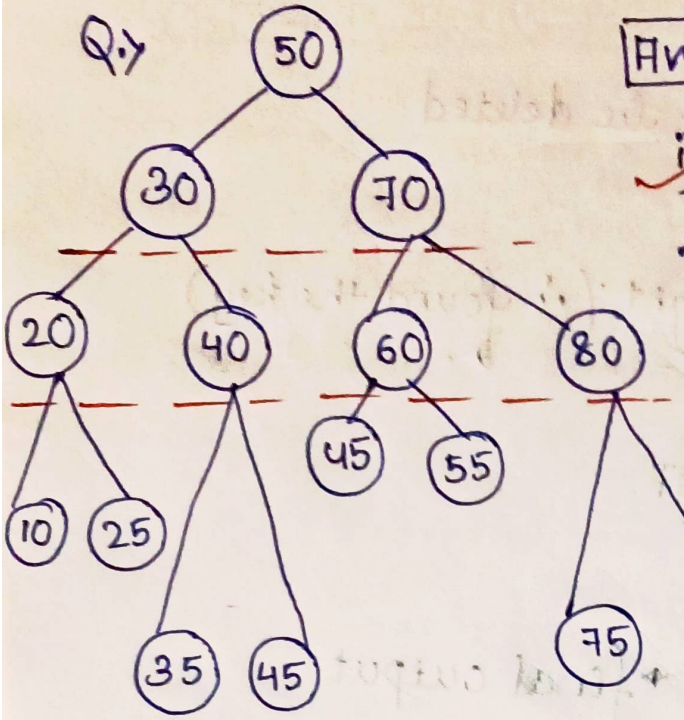
Height Formula for any Subtree for any node :-

↳ max (height of left Subtree, hgt of right Subtree) + 1

c) $O(\log n)$ → complexity [Search, insert, delete]

Balancing AVL Tree :→

Q.)



Answer :-

i) always start from leaf nodes

ii) Height rule :-

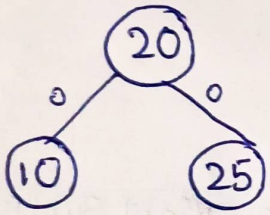
Leaf node → 0

Empty Subtree → -1

Parent of Leaf nodes → 1

→ All Leaf nodes have height = 0

Node 20



10 → Leaf node

∴ height = -1 ⇒ max(-1, -1) + 1 = 0

25 → Leaf node

∴ height = -1 ⇒ max(-1, -1) + 1 = 0

∴ Height of 20 ⇒ max(0, 0) + 1 = (1)

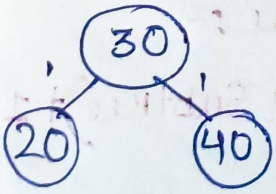
Balance Factor of 20 ⇒ 0 - 0 = 0

Node 40 ⇒ Height ⇒ max(0, 0) + 1 = (1)

Node 60, 80 ⇒ Height ⇒ (1)

Node 30 ⇒ tree is Balanced, hence

∴ Every time you more upward add 1



⇒ Height ⇒ (2)

BF ⇒ 1 - 1 ⇒ 0

Node 70 ⇒ Height = (2)

BF = 0

Root 50

Height ⇒ 2 + 1 ⇒ (3)

BF ⇒ 2 - 2 = 0

∴ It's a balanced AVL Tree.

Rotations :→ [Rotations are always done within 3 nodes]

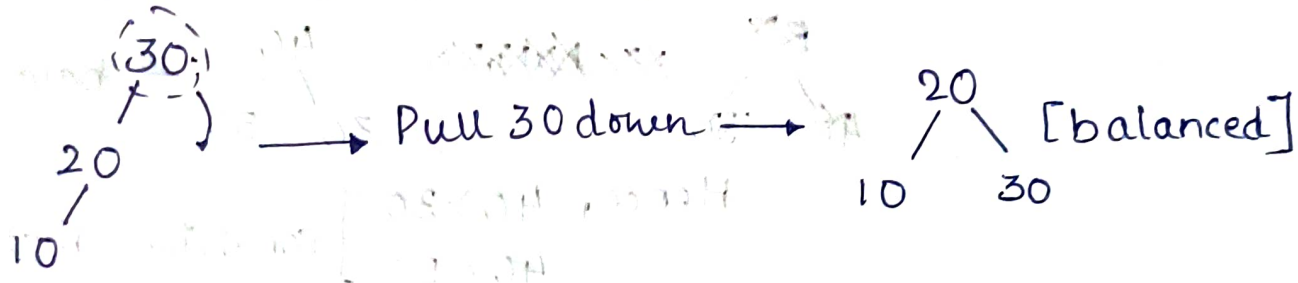
1.) LL (Left Left) → Rotate to right

2.) RR (Right Right) → Rotate to left

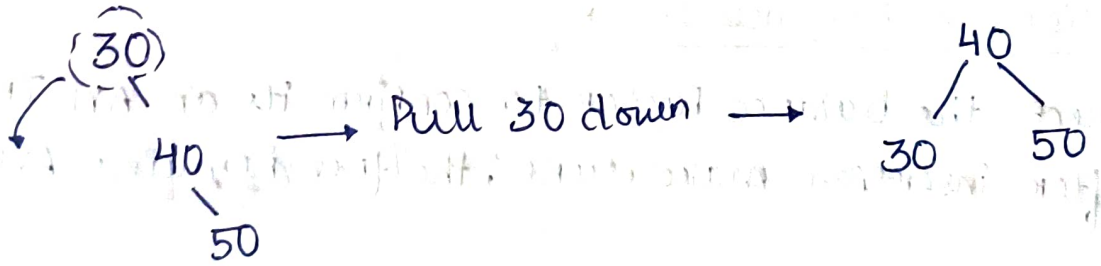
3.) LR (Left Right) → Left Rotate then Right Rotate

4.) RL (Right Left) → Right Rotate then left Rotate

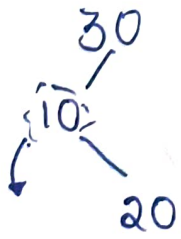
1.) Left Left Rotation :→



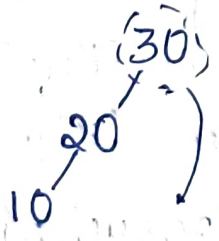
2.) Right Right Rotation :→



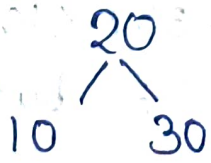
3.) Left Right Rotation :→



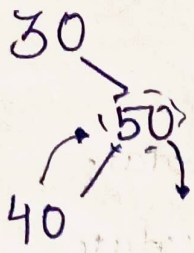
Step 1:- Rotate on 10 for Left, Pull down



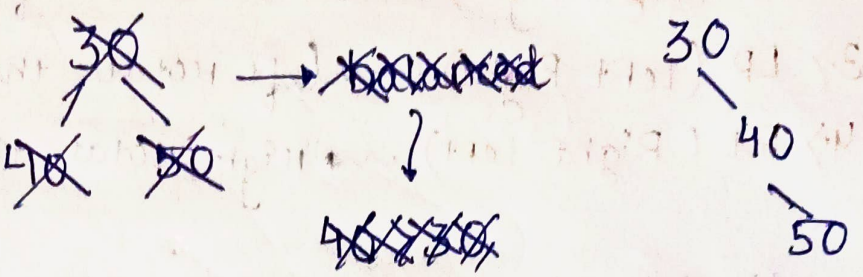
Step 2:- Rotate on 30 for Right, pull down



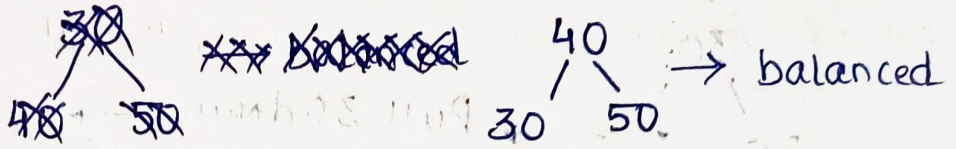
4.) Right Left Rotation :-



Step 1 :- Rotate on 50 for ~~left~~ right, pull down



Step 2 :- Rotate on 30 for left, pull down



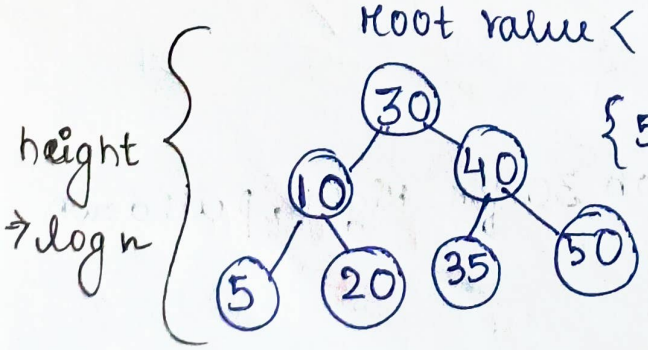
Hence, $40 > 30$] Conditions matched.
$40 < 50$]

Insertions in AVL Tree :⟶

1. Check the balance factor to confirm its an AVL Tree or not
2. After insertion make sure, the tree remains AVL.

Q.) Create a Binary Search Tree
inserting elements ⇒ [30, 40, 10, 50, 20, 5, 35]

Ans.) Check every element from root
root value > element [Left]
root value < element [Right]

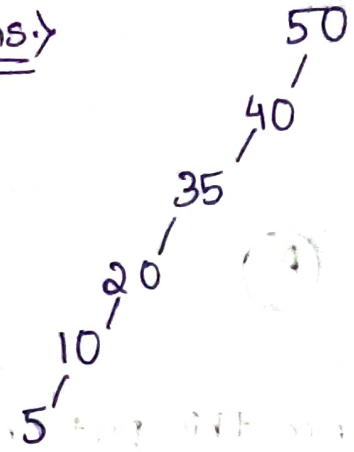


{50 > (30 & 40) ∴ Right}
{20 < 30 → Left but 20 > 10 ∴ Right of 10}
{5 < 30 & < 10 ∴ Left of 10}
{35 > 30 but 35 < 40 ∴ Left of 40}

Q.) Create a Binary Search Tree inserting elements

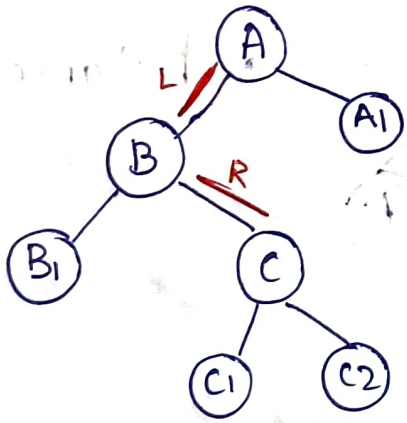
⇒ [50, 40, 35, 20, 10, 5]

Ans.)



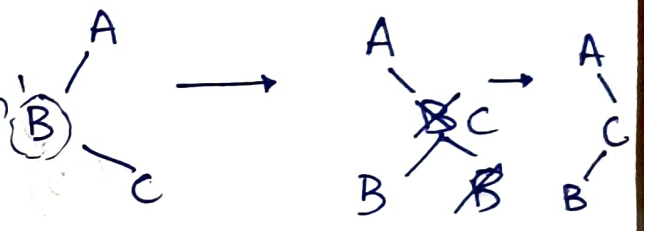
∴ height = n

Q.) Perform a Left Right rotation on a given BST to balance.



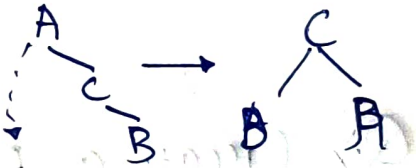
Step 1:

Pull B down



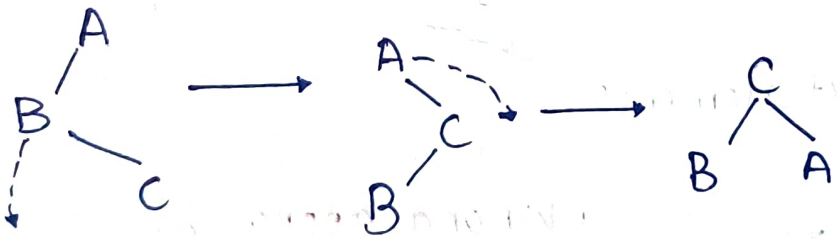
Step 2:

Pull A down

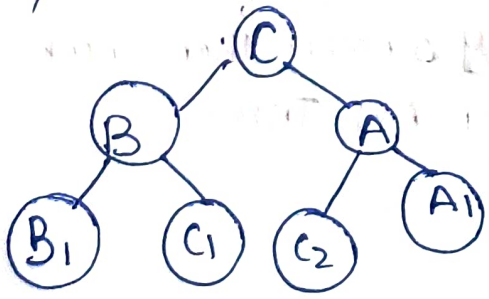


i) Left Rotate then Right Rotate

⇒



ii) Then, attach child nodes,



↓

balanced.

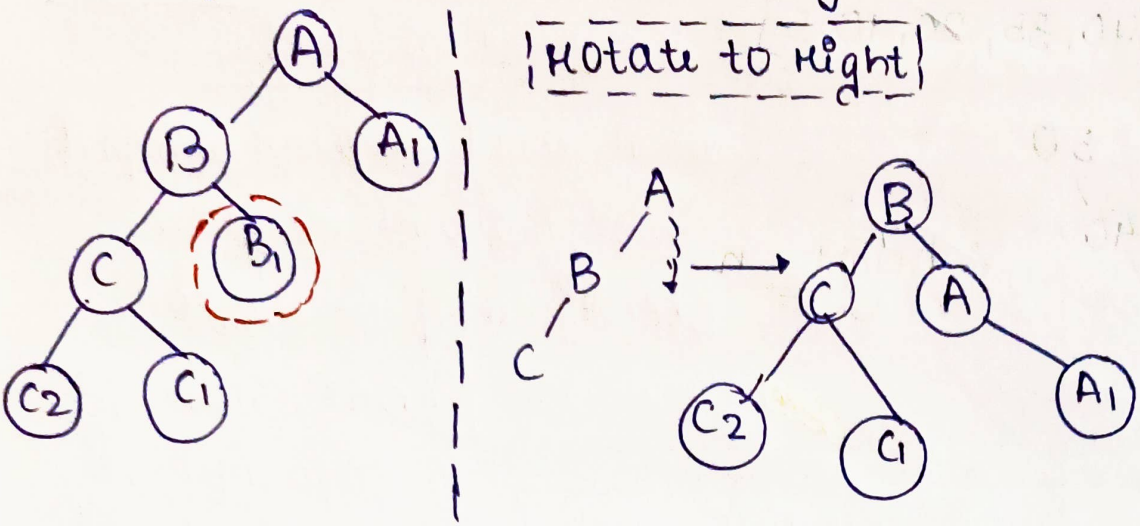
[A_1 on A's right as it is]

[B_1 on B's left as it is]

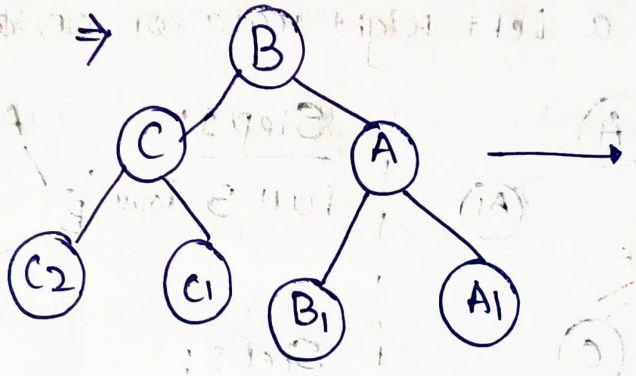
[C_1 is left of C ∴ go left of 'C' root node to balance put on B's right]

[C_2 is right of C ∴ go right of C root node to balance attach left of A]

Q.) Perform a LL rotation on a given BST to balance.



: missing B_1, to balance this put on A's left child

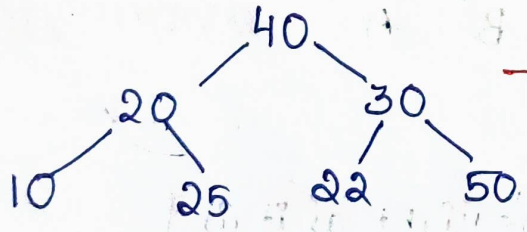


→ balanced.

Q.) Create a AVL Tree on given elements by inserting them

⇒ [40, 20, 10, 25, 30, 22, 50]

Step 1: Insert elements



→ Wrong approach.

Rotations are performed on every 3 nodes then check BF for AVL Tree.

→ i) 40 (2)

1 20

0 10

0

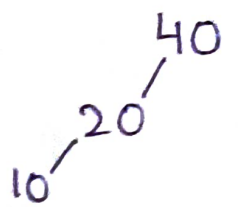
Check BF

10 = 0

20 = 1-0 = 1

40 = 2-0 = 2

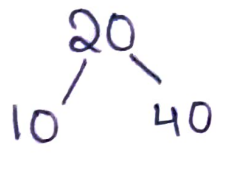
Step 2: 40 became unbalanced.



→ Perform LL rotation as its left subtrees

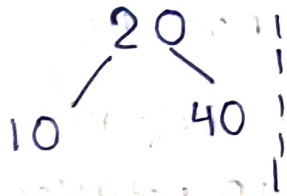
Step 3: Pull 40 right for LL rotation

→



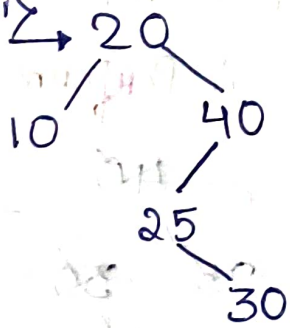
Check BF
10 ⇒ 0
40 ⇒ 0
& 20 ⇒ 1-1 = 0 [Balanced]

Step 4:



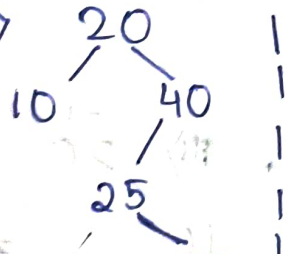
attach 25, 30

ii) →



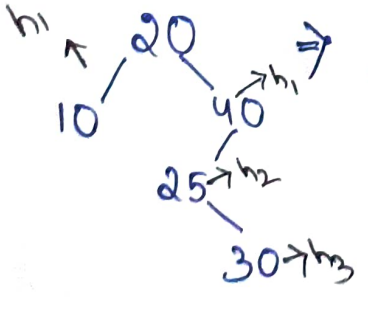
Step 5: Check BF

ii)



30 ⇒ 0
25 ⇒ 0-1 ⇒ -1
40 ⇒ 1-(-1) ⇒ 2 [∵ empty height = -1]
10 ⇒ 0 (∵ leaf node)
20 ⇒ [height of left subtree - height of right subtree]
⇒ 0 - [Level 0 + Level 1 + Level 2 + Level 3]
⇒ 0 - [Total 3 levels]

There are 3 height subtree

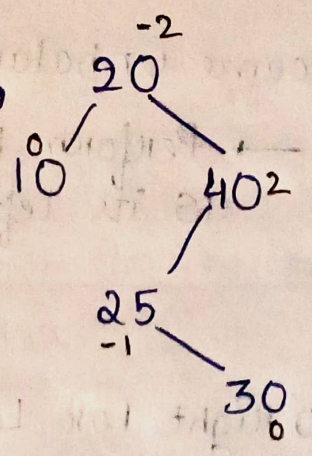


⇒ 1-3 ⇒ -2

∴ BF,
30 = 0
25 = -1
40 = 2

10 = 0
20 = -2
Unbalanced Tree.

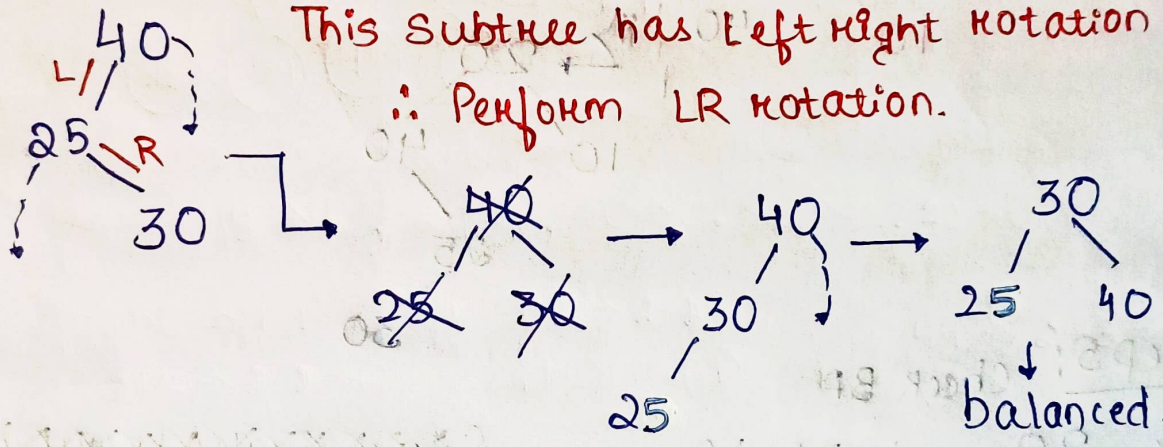
Step 6: Tree is unbalanced, Perform rotations



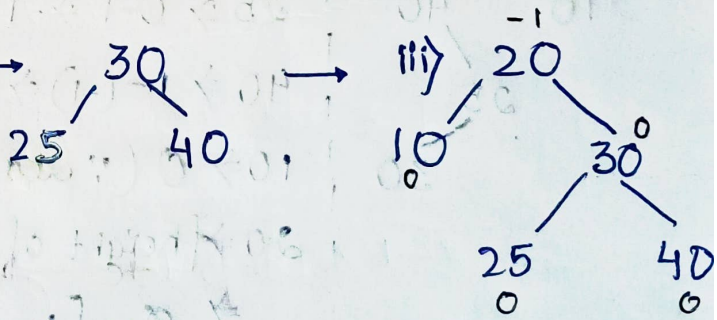
Perform rotation from there where the node became unbalanced. from checking BF's when moving downwards to upwards.

Hence, 40 became unbalanced.

Step 7: Perform rotation from node 40.



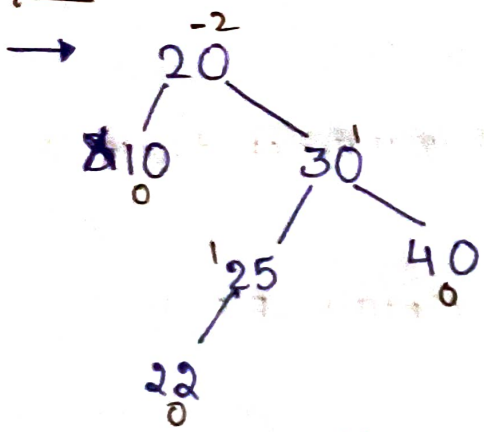
Step 8: update the subtree and check BF



BF's :→ 10 ⇒ 0
25 ⇒ 0
40 ⇒ 0
30 ⇒ 1-1 = 0
20 ⇒ 1-2 = -1

Hence, balanced. ∴

Step 9: Attach remaining elements 22 on the updated BST.

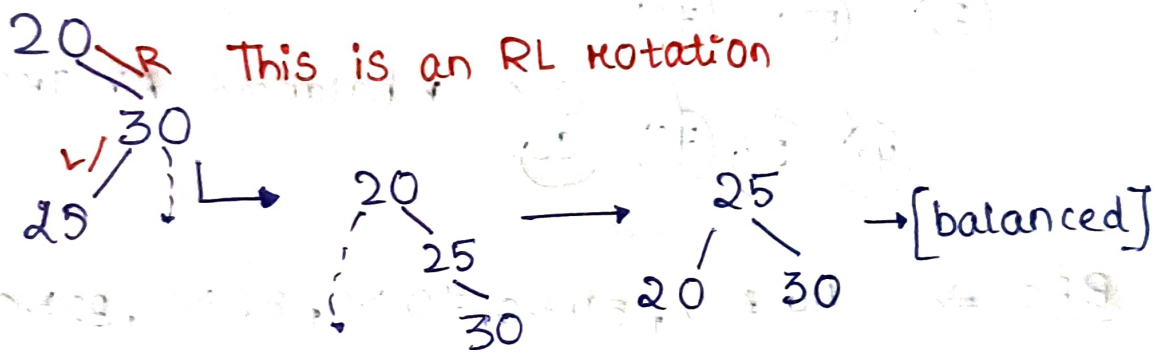


Update BF's

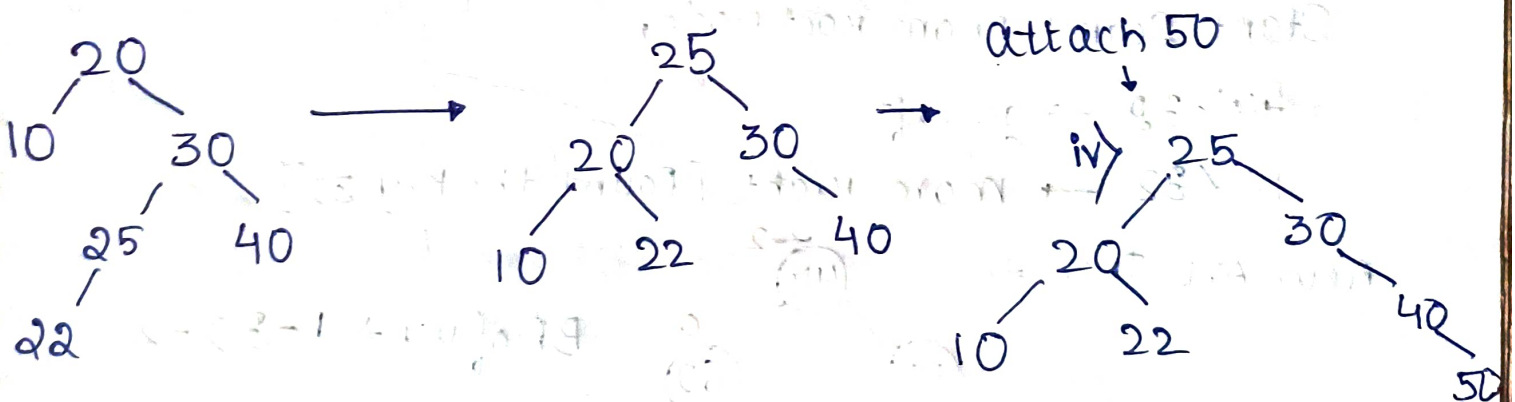
→ 22 ⇒ 0
10 ⇒ 0
40 ⇒ 0
25 ⇒ 1-0 = 1
30 ⇒ 2-1 = 1
20 ⇒ 1-3 ⇒ -2

Hence, 20 became unbalanced, Perform rotations.

Step 10: Check from 20 to perform rotation.



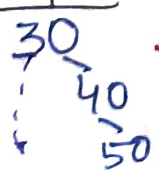
Step 11: update the subtree and attach last element.



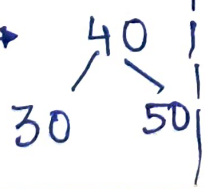
Check BF's ⇒ 10, 22, 50 = 0
40 ⇒ 0-1 = -1
30 ⇒ 0-2 = -2
20 ⇒ 1-1 = 0
25 ⇒ 2-3 = -1

Hence, 30 became unbalanced.

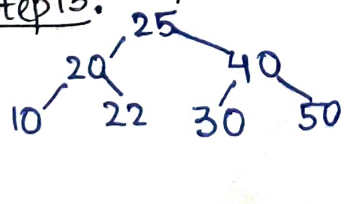
Step 12: Perform rotation on 30



→ RR Rotation →



Step 13: Update the tree.

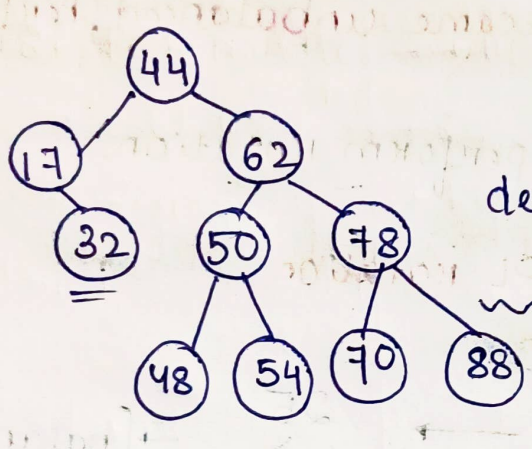


} final balanced AVL Tree.

Deletion in AVL Tree :⟶

1. After deletion of nodes check the BF whether the tree maintains AVL properties or not.
   if not perform rotations to maintain BF.

Q.)



delete input key ⇒ 32

⟿ balanced AVL Tree.

BF's ⇒ 48 ⇒ 0, 54 ⇒ 0, 70 ⇒ 0, 88 ⇒ 0, 32 ⇒ 0

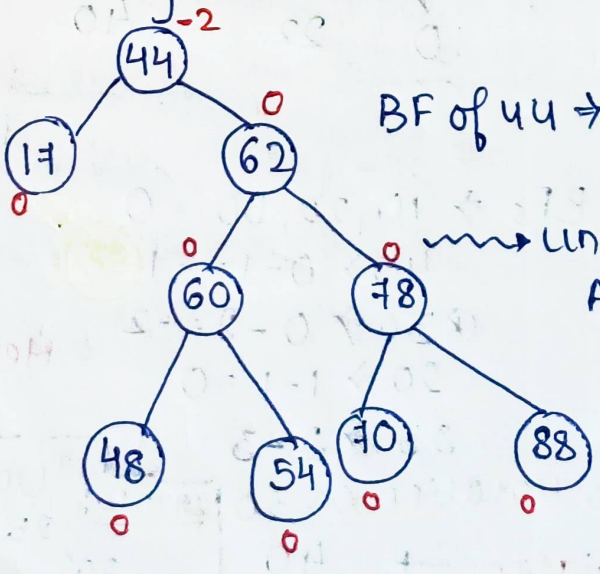
50 ⇒ 1-1 = 0 ; 62 ⇒ 2-2 = 0 ; 44 ⇒ 2-3 = -1
78 ⇒ 2-2 = 0 ; 17 ⇒ 0-(-1) = -1 ; 60 ⇒ 0

Start Search From root node,

44 > 32 → go left

17 < 32 → more right [Found the key 32]

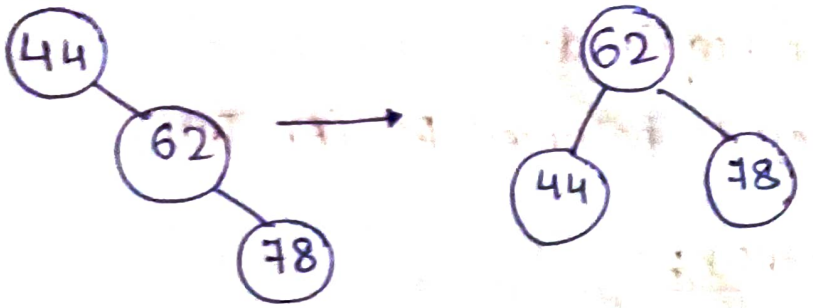
new AVL Tree ⇒



BF of 44 ⇒ 1-3 ⇒ -2

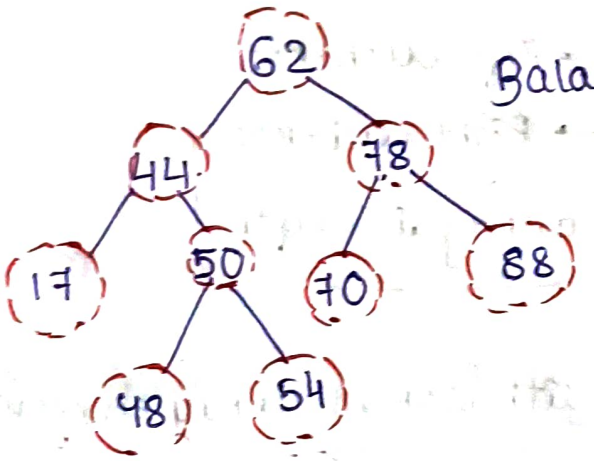
⟿ Unbalanced AVL Tree.

Now, perform RR rotation to balance the tree.



∴ The New balanced Tree is

→



Balanced AVL Tree.

Weak AVL | Rank Balanced Trees :→

i) Weak AVL Trees are also known as rank balanced Trees

ii) Self balancing BST

iii) aims to balance the tree while minimizing no of rotations.

iv) are less balanced leading to longer search times.

v) Height of tree ⇒ O(Log n)

vi) Insertions and deletion operations in worst case ⇒ O(log n)

Optimal Sequence for AVL Tree insertions :-

- no rotations required.
- sorted array to balanced BST (AVL Tree)

memory trick:

1. Sort the array
2. Pick middle → root
3. Left half → left subtree
4. Right half → Right subtree
5. Repeat until array is empty.

Q.) Create an AVL Tree with sorted array elements.

Array = [5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75]

Ans) i) middle element = 40

[15 elements ∴ 40 is middle one]

ii) Left : [5, 10, 15, 20, 25, 30, 35] ~~40]~~

Right : [45, 50, 55, 60, 65, 70, 75].

ii) Choose the middle of each half.

Left half : [5, 10, 15, 20, 25, 30, 35]

middle : 20

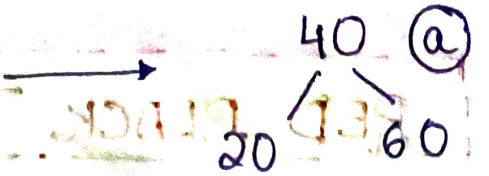
Right half : [45, 50, 55, 60, 65, 70, 75]

middle : 60

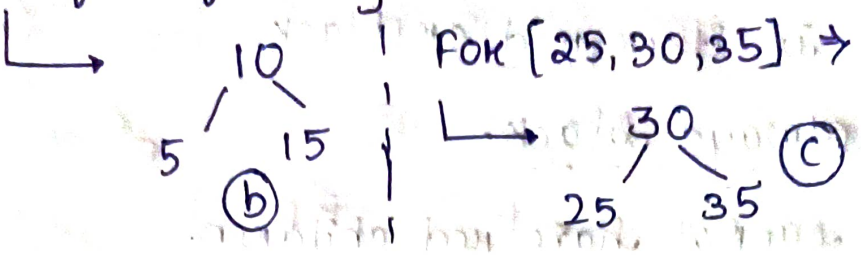
iv) For left : [5, 10, 15] 20 [25, 30, 35]

For right : [45, 50, 55] 60 [65, 70, 75]

v) Root element → 40
Left middle → 20
Right middle → 60

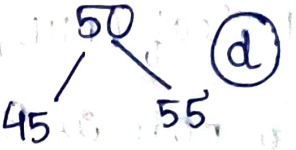


vi) For left half array the middle element is 10, [5, 10, 15]

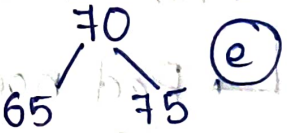


vii) For right half array the middle element is,

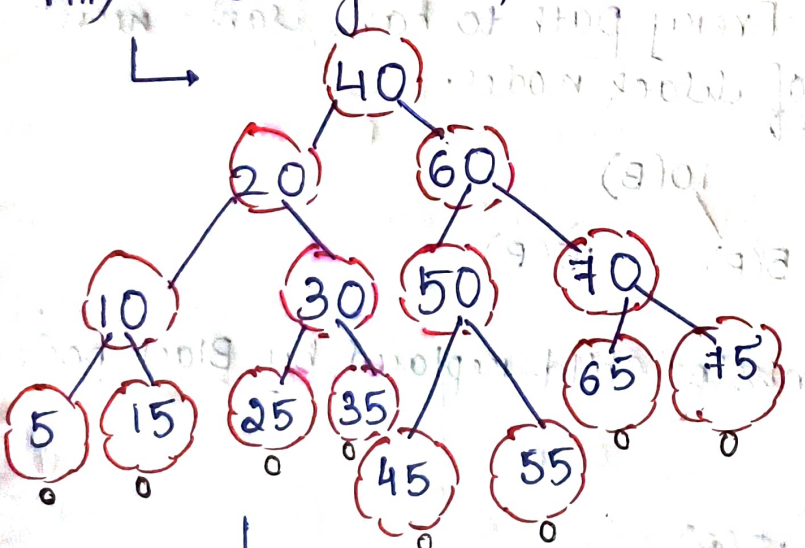
for [45, 50, 55] → 50



for [65, 70, 75] → 70



viii) Place together ; a, b, c, d, e serially.



final output.

BFS :→
5, 15, 25, 35, 45, 55, 65, 75 → 0 ✓
10, 30, 50, 70 → 1-1 → 0 ✓
20 → 2-2 → 0 ✓
60 → 2-2 → 0 ✓
40 → 3-3 → 0 ✓

Hence, balanced.

Time complexity :- O(n)

Space complexity :- O(log n).

RED BLACK TREE :→

Properties :→

1.) Each node is either red or black.

2.) Root is always black.

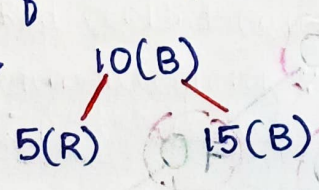
3.) Red nodes cannot have red children.
Red nodes cannot be adjacent.

4.) All leaf (NIL/NULL) nodes are black. (i.e nodes not present)

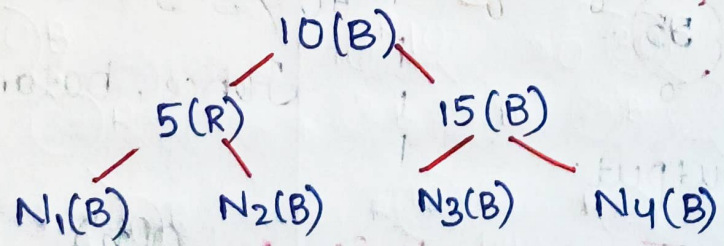
5.) If a node is Red, then both of its children must be Black.
↳ no consecutive Red nodes.
↳ Red node :→ black children.
(Red parent and Red child together not allowed)

6.) Same black height :- Every path to have same number of black nodes.

Ex:- i) Given tree →



ii) Replace every missing child replaced by Black node.



iii) Check the paths

Path 1 :→ 10(B) → 5(R) → N1(B) [2 black nodes]

Path 2 :→ 10(B) → 15(B) → N3(B) [3 black nodes]

↳ every path doesn't have same number of black nodes ∴ it's an invalid red black tree.

7.) Height of a red black tree with 'N' Nodes

$\Rightarrow$ $h <= 2\log(N+1)$

8.) Number of black nodes from root to leaf (including NULL)

$\Rightarrow$ black Height $>= h/2$

Insertion in red black tree :→

Approach :-

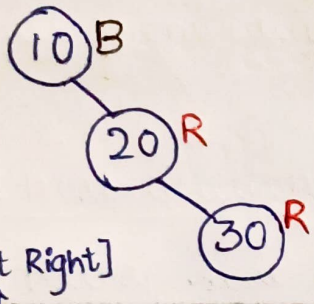
1. root node is always black.
2. All other roots initially inserted are red first.
3. If red-red conflicts occurs check these cases:
   - → If uncle node is Black or NULL, perform both rotation and recolor those nodes whose positions changed after rotation.
   - → If uncle node is red the perform these, recolor parent, uncle and grandparent node.
4. Check the tree again if any conflicts occurs, perform the actions needed according to the conditions.
5. follow the rules of recoloring but do not change the colour of root node under any circumstances.

Q.) 10, 20, 30, 15, 25, 5, 12, 35, 40, 32, 50

insert these elements to make a red black tree.

Ans) 10 → root node (Black)

i)



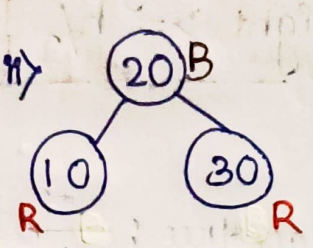
[Right Right]

Becomes RR conflict

Balance it and RR conflict

[Red Red]

ii)

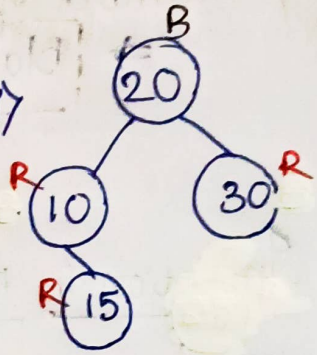


20 red → 20 black

10 black → 10 red.

RR conflict, uncle null, rotations and recolour. whose positions changed.

iii)

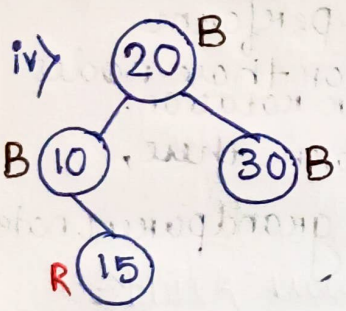


Attached elements 15

RR conflict came,

[Red Red] Recolour nodes.

iv)

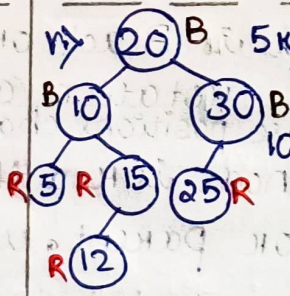


10 red → 10 black

30 red → 30 black

20 black ∵ root node

v)



5 red → 5 black

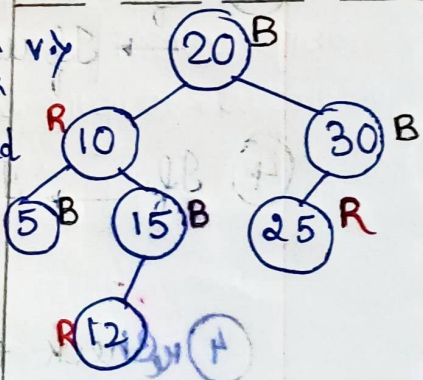
15 red → 15 black

10 black → 10 red

Red Red conflict came

Perform recolour ∵ uncle node is red.

vi)



Red Red conflict occurred

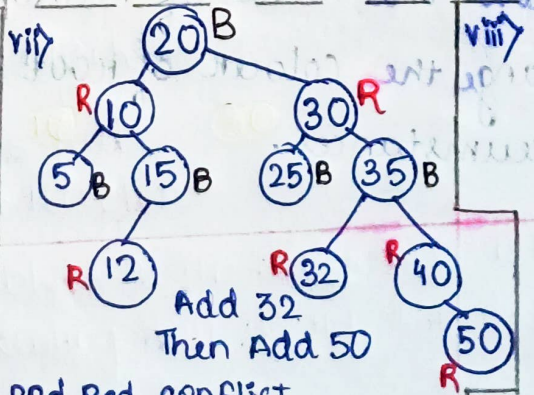
Recolour nodes as uncle node is present

25 R → 25 B

35 R → 35 B

30 B → 30 R

vii)



Add 32

Then Add 50

Red Red conflict

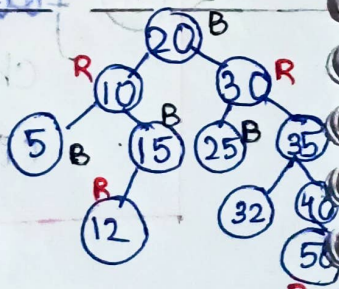
Recolour nodes

32 R → 32 B

40 R → 42 B

35 B → 35 R

viii)

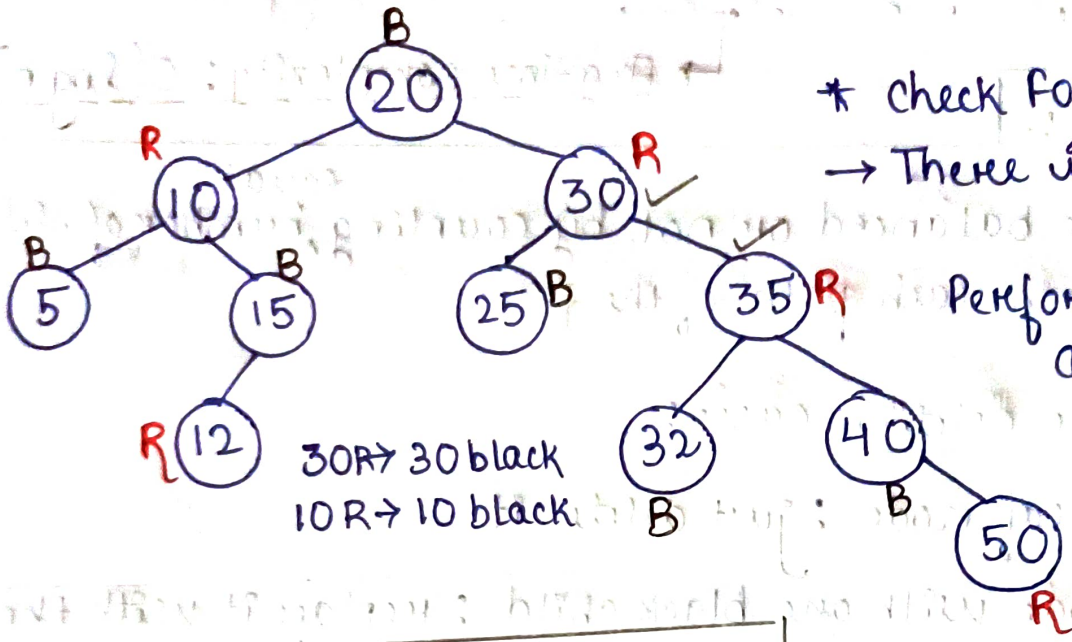


35 Red

40 black

32 black

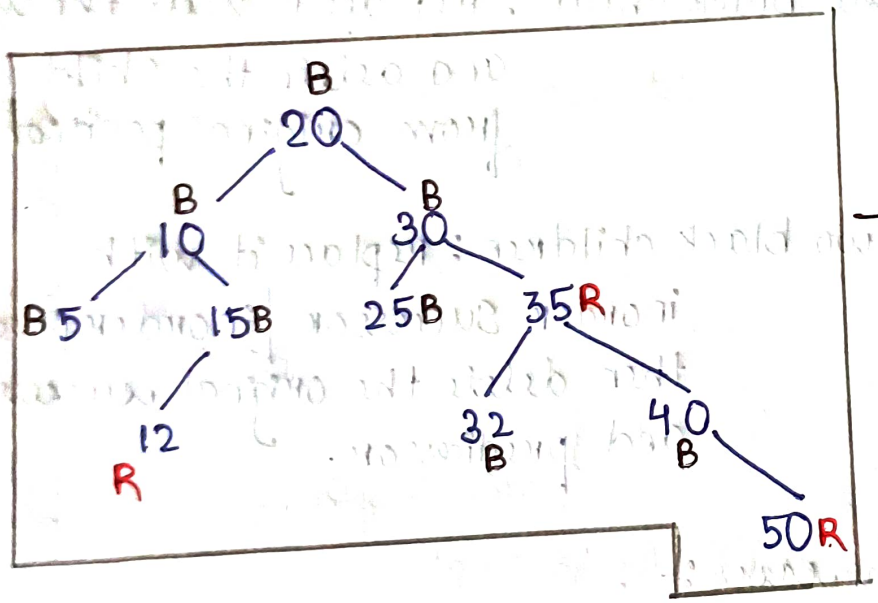
ix) balanced red black tree →



30R → 30 black
10R → 10 black

* check for any conflicts
→ There is a Red Red conflict
Perform recolour as uncle is Red

x)



→ final output
Same number of black nodes from any direction

∴ It's an red black Tree.

Deletion in red black trees:-

↳ Avg time complexity: O(log n)

Approach:-

1) Check the balanced or not by counting equal number of black nodes from each path of the tree.

2) Red node deletion cases:-

i) → Red leaf node : just delete it

ii) → Red node with one black child : replace it with the child and delete the child from original position

iii) → Red node with two black children : replace it with inorder successor / inorder predecessor then delete the original successor and predicessor.

3) black node deletion cases:-

1) Black node with 1 red child:
→ Replace with red child, color the black child

ii) Black Leaf node:
→ creates a Double black

Case 1: Sibling is red
⇒ Rotate + recolor

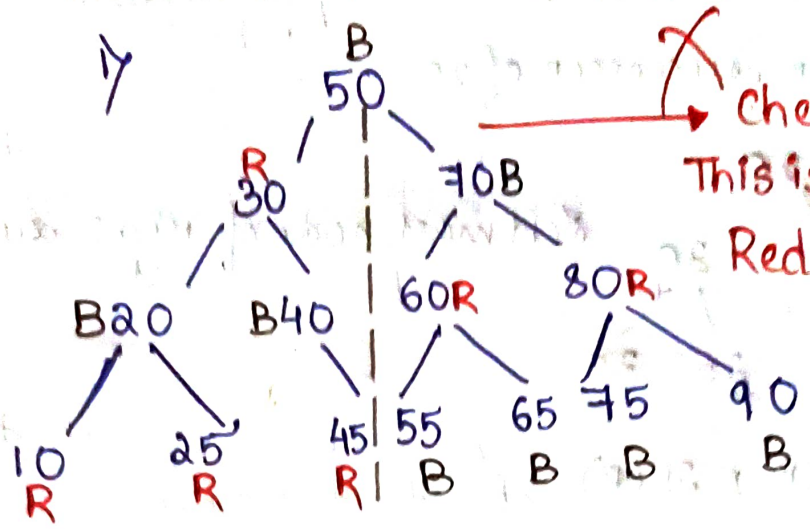
Case 2: Sibling is black with inner red child
⇒ Rotate sibling with its inner red child.

Case 3: Sibling is black with an outer red child
⇒ Rotate the parent and recolour.

Case 4: Sibling black with two black children.
⇒ colour sibling red, move double black up.

Q) Perform red node deletion operations.

1)

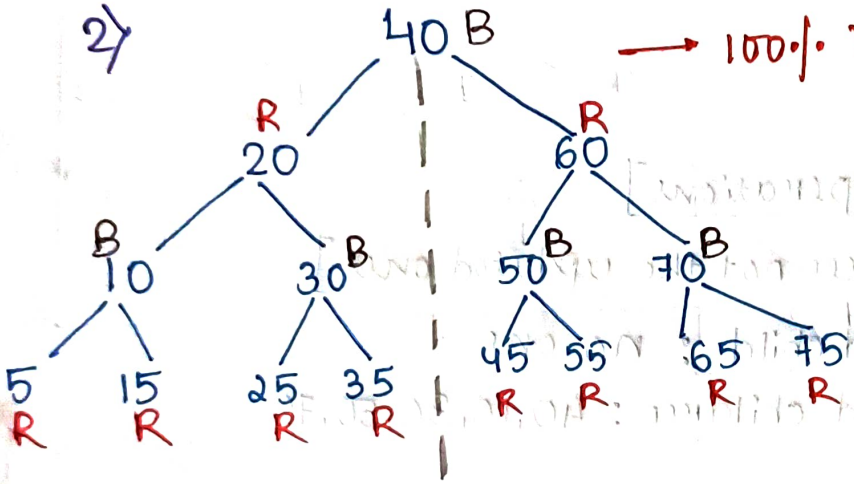


Checked
This is not a valid Red Black Tree.

(I) Read Leaf node : 10, 25, 45

(II) Red node with 1 black child : 0

2)



→ 100% valid Red black tree

(I) Red Leaf node : 5, 15, 25, 35, 45, 55, 65, 75

(II) Red ~~Leaf~~ node with 2 black child : 10, 30, 50, 70

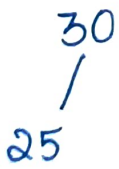
(III) ~~No~~ Red node with 1 black child : no one.

[Red node deletion operations]

1) delete key = 35

→ normal deletion followed

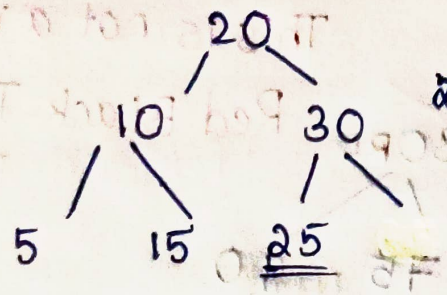
→



ii) delete key = 20

replace it with inorder successor / predecessor

→ find inorder successor of 20

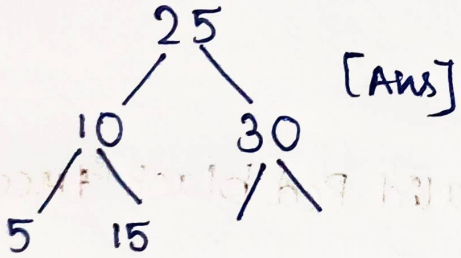


leftmost node of right subtree

→ ∴ successor = 25 (R)

→ Replace 20 with 25 then delete 25 from initial position.

→



[Ans]

[Black node deletion operations]

[use the original tree not the updated ones]

(I) Black node with 1 red child : no one

(II) Black node with 2 red children : 40, 10, 30, 50, 70

(III) Black leaf children/nodes : no one

(IV) Black leaf with red sibling : no one

(V) There are no black leaf nodes ∴ double black operations cannot be performed.

B+ TREE :→

Properties :→

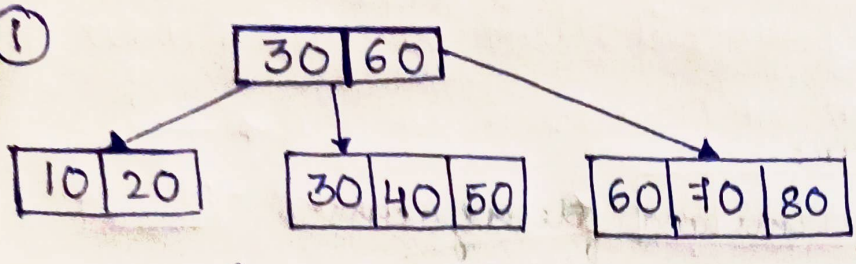
i) advance data structure
ii) used in database systems and file systems.
iii) maintains sorted data for fast retrieval.
iv) extended version of B tree. [actual data only stored in leaf node & internal nodes contains only keys for navigation]
v) Leaf node stores all the key values and pointers to the actual data.
vi) Internal node stores only the keys that guide searches.
vii) all Leaf nodes are Linked together for efficient range queries.

→ This structure makes the B+ tree :-
- i) balanced
- ii) disk-efficient
- iii) ideal for database indexing:

viii) Balanced, multi-Level, Ordered, Cache-Friendly for better speed.
ix) Each node has many children, keeping the tree short & fast.

Searching in B+ Tree :-

(1)

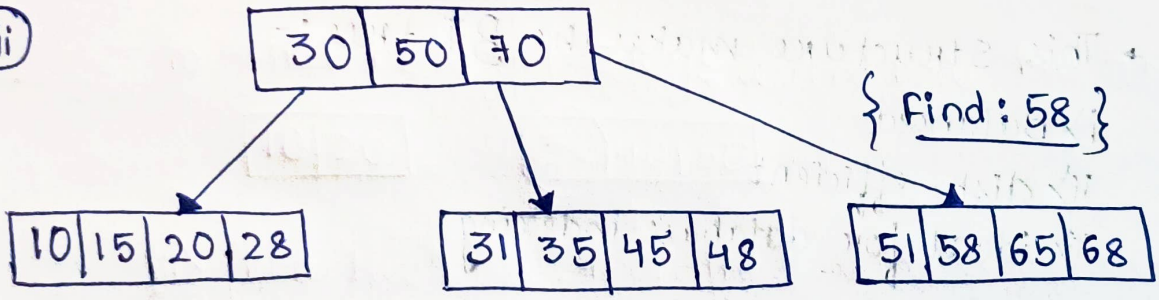


{ Find : 70 }

Approach:-

i) Start at the root
   ⇒ 30|60 → 70 > 60 → go right
ii) We reached 80
iii) We found 70, just beside 80 [Search over]

(ii)



{ Find : 58 }

Approach :-

→ ~~5 20 30 58~~ 58 > 30, 58 > 50, 58 < 70 [Travel right]

→ Now we are at [51, 58, 65, 68]

→ Inside this, 58 is found.

Insertion in B+ tree :-

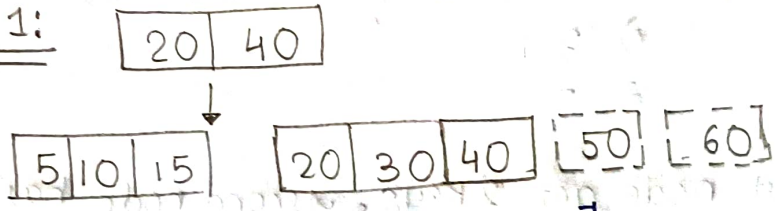
→ Insertion always happens at the leaf level.
→ but if a node becomes full, we split it and adjust
→ A node can contain a maximum of 4 keys.
→ A node must have atleast 2 keys (except the root)
→ If a node gets 5 keys, it overflows.

* Order 5 = max 4 keys.
  Fifth key causes overflow.

Case 1: → Overflow in Leaf node

→ if a leaf node contains morethan 4 keys, we divide it into two leaf nodes.

Ex 1:

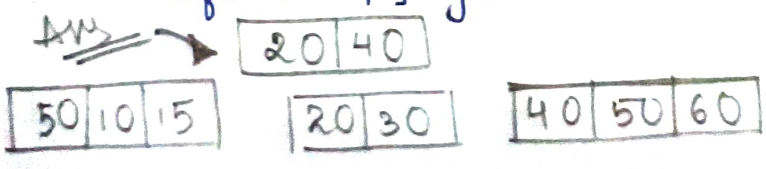


i) Insert 50 [∵ max is 4 keys]
ii) Insert 60 [Overflows]
iii) as overflow occurred, split the Leaf node.
iv) as per Formula, First node ⇒ $[(M-1)/2]$ $\{M=5\}$
⇒ $[(5-1)/2]$
⇒ $[4/2]$
⇒ 2

v) first node get 2 keys
vi) second node gets remaining 3 keys.



vii) COPY the smallest key of the second node to the parent. after copying, it also remains in the Leaf



Right biased splitting.
→ extra key goes to right node.

Case 2:→ Overflow in Non-Leaf node (internal node)

→ Sometimes, splitting a Leaf adds a new key to its parent. What if the parent also becomes full?

!
!—→ Then we must split the internal node too.

Ex 1:- Internal node ⇒ | 10 | 20 | 30 | 40 |

| 10 | 20 | 30 | 40 | 50 | {50 comes from a split child}

→ Overflow.

{ max keys allowed = 4 }

i) Perform, $[M/2] - 1$ {M = 5}

⇒ $(5/2) - 1$

⇒ $(2.5) - 1$ (round 2.5 → 3)

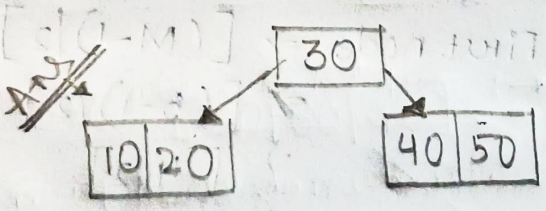
⇒ 3 - 1

⇒ 2

ii) First node get 2 keys, second node has 3 keys

| 10 | 20 | | 30 | 40 | 50 |

iii) from two nodes, more the middle separator key as parent



iv) separator keys moves as parent but doesn't remain in its original Leaf.

Difference b/w case 1 and case 2 :→

i) In Leaf node splitting, Separator key is copied to the parent and remains in the Leaf as well.

ii) In non Leaf (internal) node splitting, separator key is copied to parent but doesn't remain in its orginal place.

iii) Root overflows → create new root

↓

height ⇒ height + 1

iv) Case 1 [Leaf node] → First node → $[(M-1)/2]$

v) case 2 [non Leaf node] → First node → $[(M+2)-1]$

vi) Case 1 [Leaf node] → Smallest key → copied upward to right

vii) case 2 [non leaf node] → separator middle key → as parent.

Deletion in B^+ tree :→

→ When compared to the B tree, B^+ tree offers more effective insertion, deletion and other operations.

→ Involves three operations
- Searching
- Deleting
- Balancing.

→ while deleting maintain the properties of B^+ tree.

i) Actual keys are stored in leaf nodes
ii) All leaf nodes must remain at same level.
iii) Leaf node must contain the minimum required no of keys.
iv) Leaf nodes are connected to each other.

→ max keys = 4 , min keys = 2
If a leaf has fewer than 2 keys, it has underflow.

→ Simple deletion ⇒ no underflow
Borrowing ⇒ take a key from a sibling
Merging ⇒ Combine two nodes when borrowing is not possible

steps to check / create a B^+ tree :→

i) Leaf nodes must be at same level.
ii) Internal nodes can have at most 'M' children
→ can have at least 'M/2' children.
iii) Actual data records are stored at the leaf nodes.
iv) leaf nodes are linked sequentially for fast scanning.
v) All keys inside every node are strictly maintained in ascending order.
vi) Internal nodes store only index keys to guide search paths, not actual data.

Ex:- [N | 14 | 20]

[1|3|5] [11|12] [14|15|17]

[1|2] [3|4] [5|7] [11] [12] [14|15]

[delete key = 5]

Ex:- [40] → valid B+ tree

[20|30] [50|60]

[10|15] → [20|25] → [30|35] → [40|45] → [50|55] → [60|70]

40 → Root node

[20|30] [50|60] → Internal nodes. [2 keys, 3 children]

[20|30] → 20 & 30 are the keys

children ↑ [20 | 30] children

[10|15] [20|25] [30|35]

↳ children

[50|60]

[40|45] [50|55] [60|70]

↳ children children children

---

[40]

Order ⇒ 3 (M)

So, min key ⇒ [M|2] - 1 in a leaf ⇒ (3|2) - 1 ⇒ 1 key

[20|30] [50|60]

[10|15] [20|25] [30|35] [40] [50|55] [60|70]

Simple deletion:→ delete key ⇒ 15

i) search 15 → [10|15]

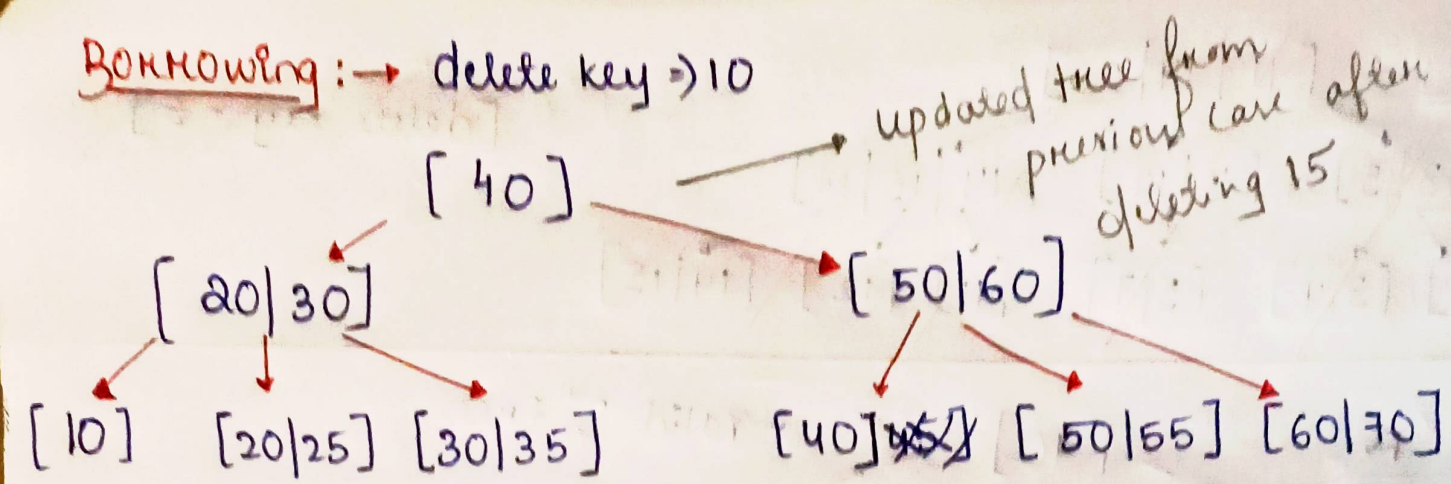
ii) delete 15

iii) Now only 10 left (as per order it can have 1 key)

iv) ∴ No underflow.

v) 15 was not an internal key, ∴ no changes needed.

Borrowing :→ delete key → 10



i) 10 < 40 [go left]
ii) 10 < 20 [go further left]
iii) 10 found
iv) Delete
v) underflow occurred (must have 1 key at least)
vi) check right sibling [20|25] {can lend 1 key}
vii) Borrow the smallest one → 20
viii) updated tree →

